Hana Matsuri: Festival After Dusk — Assignment 3: MVP Implementation

CS549 Artificial Intelligence for Visual Computing · Shweatha Bathina Mallikarjuna (sb2808) · Chiranjeevi Bhuvaneshwari Pamarthy (cp1270) · Jayanth Jayaram Jayakumar Naidu (jj1112) · Prithvi Govindareddy (pg708) · Mentor: Danrui Li

Playable build	https://hana-matsuri.vercel.app
Demo video	(YouTube link — added at submission)
Source code	https://github.com/Shwethabathina/hana-matsuri (README.md has build instructions)

1. Plan vs. Reality

Our Assignment 2 MVP was a fully playable **Goldfish Scooping (Kingyo-sukui)** activity inside the festival world: independently moving fish, a responsive paper scoop, fish–scoop collision, real-time water feedback without a full fluid solver, and a result that returns to the festival HUD. The success test (walk to stall → press E → control scoop → water reacts → catch fish → reward and objective complete) works in one continuous session in the live build.

Made it in (MVP)

- **Animated fish:** a rigged goldfish glTF (8 clips) cloned per fish with its own `AnimationMixer`; slow/fast swim cycles blend by speed and the *jump* clip plays when a fish is scooped.
- **Fish behaviour:** boids (separation, alignment, cohesion) + tub-wall avoidance + fleeing the submerged scoop; fish change depth and dive when frightened.
- **Water:** height-field simulation (discrete 2D wave equation, 96×41 grid, fixed 60 Hz step, damping, reflecting borders). The scoop, splashes, caught fish and shallow fish disturb the surface; normals and crest colours are rebuilt each step.
- **Collision:** a fish is caught only if its body circle overlaps the paper disc *and* it is above the paper's depth when lifted. Paper strength drops with water drag (scoop speed) and fish weight; the paper visibly tears. Debug view (V) draws the catch radius and each fish's hit circle.
- **Game loop and tooling:** yen reward, prize, objective ticked, firework pattern unlocked; live FPS/fish/grid/sim-cost readout and a stress mode (P adds 20 fish, O raises water resolution).

Added beyond the MVP (scene polish)

- Food and goods on every stall counter, placed from the counter geometry of the street model; visitors walking leashed dogs (four-leg trot gait, sitting, tail wag); visitors rebuilt as single skinned meshes in yukata.
- Cherry-blossom grove and forest from CC-BY models, instanced, with wind sway in the vertex shader (height-weighted bend, gusts, leaf flutter, per-tree phase).

Cut or changed

- Water has no true refraction of the fish below it (translucent reflective surface instead). Fish collision uses 2D circles plus a depth test rather than mesh tests: cheaper and easier to read on screen.
- The goldfish material's transmission was disabled (it forced an extra full-scene render pass each frame).
- Adding the trees made the game stutter, so we did a performance pass (below). The street model was simplified from 804k to 188k triangles and two dense foliage clumps were replaced by cherry trees; the forest is sparser than first built.

Performance

Goldfish MVP (Chrome, desktop)	Fish	Water grid	Water sim	FPS
Normal play	16	96 × 41	≈1.3 ms/frame	60
Stress test	216	384 × 164	≈14 ms/frame	≈31

Stress-test costs are skinned-mesh animation (one mixer per fish) and the CPU wave simulation, which grows with grid cells. After the scenery was added we profiled triangles, draw calls and per-system CPU time and cut the frame cost from about **6.3M to 1.3–1.9M triangles per frame**: tree instances grouped into 40 m blocks for frustum and

distance culling; street and tree meshes simplified with meshoptimizer; a pool of the 4 nearest lantern lights instead of 14 point lights; cheaper Lambert shading on overlapping foliage cards; conifers excluded from the shadow map; NPCs as one skinned mesh each. Live FPS is visible in the demo video (N key).

2. AI Utility

- **Claude Code** was the most useful tool. It produced working first versions of large systems quickly (boids, height-field water, behaviour-tree NPCs, A* navigation, particle fireworks, asset compression with gltf-transform) and ran the game in a headless browser to screenshot it, catch runtime errors and measure triangles, draw calls and CPU time per system.
- **ChatGPT** helped with debugging questions, Three.js API details and shader/particle ideas. **AI image generation** was used only for concept mockups. All 3D assets in the build are CC-BY Sketchfab models or procedural geometry, not AI-generated.
- **Time saved vs. debugging:** AI saved a lot of time on first drafts, but integration and tuning took real effort. Examples: night lighting was first over-exposed; default model compression merged meshes and broke our collision lookups; some AI-proposed performance fixes hurt the experience and were reverted after playtesting (adaptive resolution made the image blurry, throttled shadow updates caused judder, removing daytime bloom changed the art direction). Human judgement on look and feel was essential.
- No runtime AI API is used, as stated in our tech spec, so the build is predictable and works offline.

3. Roadmap to Final

- **Main activities:** polish the Shooting Gallery and Taiko rhythm game and finish Kimodameshi (currently playable prototypes).
- **Characters:** replace procedural visitors with rigged human models (walk/idle blending), keeping the behaviour-tree AI.
- **Rendering:** island environment with reflective water and the raised shrine terrace (drafted in `island.js`); denser trees using impostors/LOD; caustics in the goldfish tub.
- **Gameplay:** ring toss, yo-yo fishing, mask memory, Daruma balancing and a lantern puzzle (scope permitting); fireworks that reflect more of the player's progress; GPU water and instanced fish for lower-end laptops.